

Documentation detaillee du projet

TIPE 13530 — Optimisation des tournées de collecte

Comprendre chaque fichier, chaque classe et chaque fonction du programme — meme sans savoir coder.

Chaque fonction est expliquee par : a quoi elle sert, ce qu'on lui donne (entree), ce qu'elle renvoie (sortie), et une explication en langage courant.

0. A lire d'abord (si vous ne codez pas)

Ce document explique un PROGRAMME. Pour le comprendre, six mots suffisent. On les illustre avec l'image d'une cuisine.

- Un PROGRAMME est comme un grand livre de recettes : une suite d'instructions qui transforment des ingrédients en plat.
- Un FICHIER (ou module) est un chapitre du livre : il regroupe des recettes qui vont ensemble (ex. `tsp_core.py` = le chapitre des algorithmes).
- Une FONCTION est une recette précise : on lui donne des ingrédients, elle rend un plat. Ex. 'longueur d'un tour' : on lui donne un trajet, elle rend sa durée totale.
- Un PARAMETRE (ou argument) est un ingrédient qu'on fournit à la recette.
- La VALEUR DE RETOUR est le plat fini que la recette rend à la fin.
- Une CLASSE est un APPAREIL de cuisine (ex. un robot) : un objet qui retient un état (ce qu'il contient) et possède des boutons (ses METHODES). Ce projet n'a qu'une seule classe, `Doc`, expliquée en section 9.

Trois structures de données reviennent souvent :

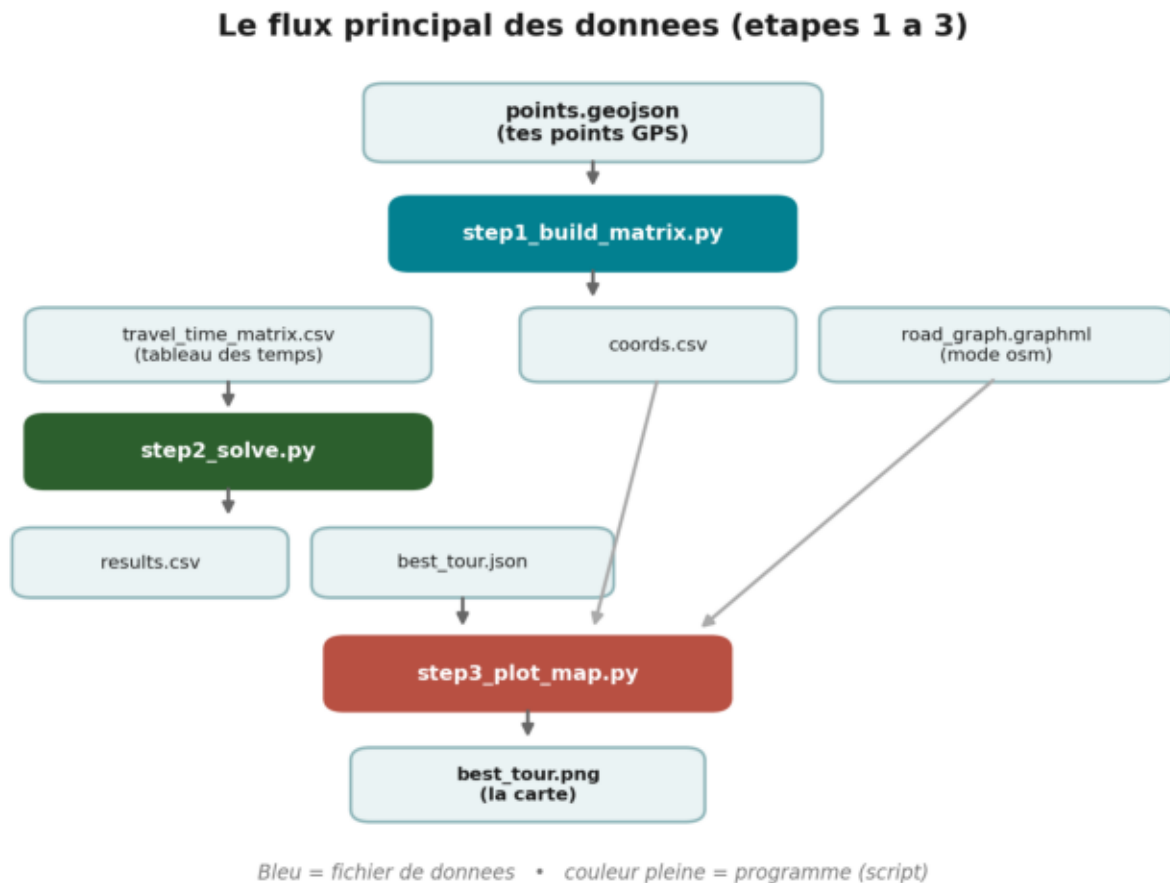
- une LISTE = une liste de courses ordonnée : [A, B, C]. On peut lire le 1er, le 2e élément, etc.
- un DICTIONNAIRE = un meuble à tiroirs étiquetés : à l'étiquette 'temps' correspond un contenu. On range et on retrouve par étiquette.
- une MATRICE = un tableau à double entrée, comme la table des kilométrages d'une carte routière : la case (ligne i, colonne j) donne le temps pour aller du point i au point j.
- un FICHIER CSV = un tableur (type Excel) en texte simple : des lignes et des colonnes séparées par des virgules.

1. Vue d'ensemble du projet

Le but : partir d'une liste de points à collecter et trouver la tournée (le cycle) la plus rapide qui les visite tous puis revient au dépôt. Le projet enchaîne des étapes, chacune dans son fichier ; chaque étape écrit des fichiers que la suivante relit.

Le schéma ci-après montre le flux principal : un fichier de points entre, une carte du meilleur trajet sort.

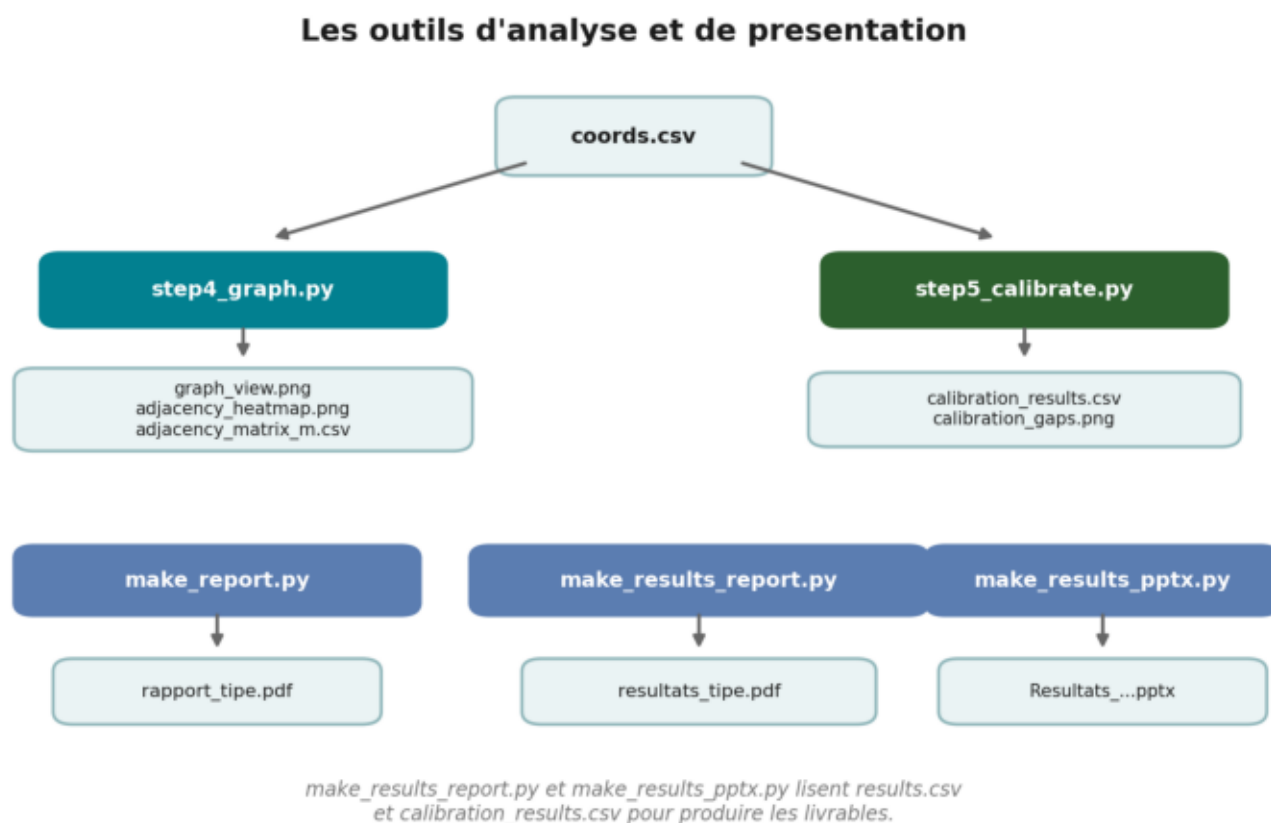
Schema 1 — Le flux principal des donnees



On lit de haut en bas : points.geojson -> step1 fabrique la matrice des temps -> step2 calcule le meilleur tour -> step3 le dessine. Les boites bleues sont des fichiers, les boites colorees sont des programmes.

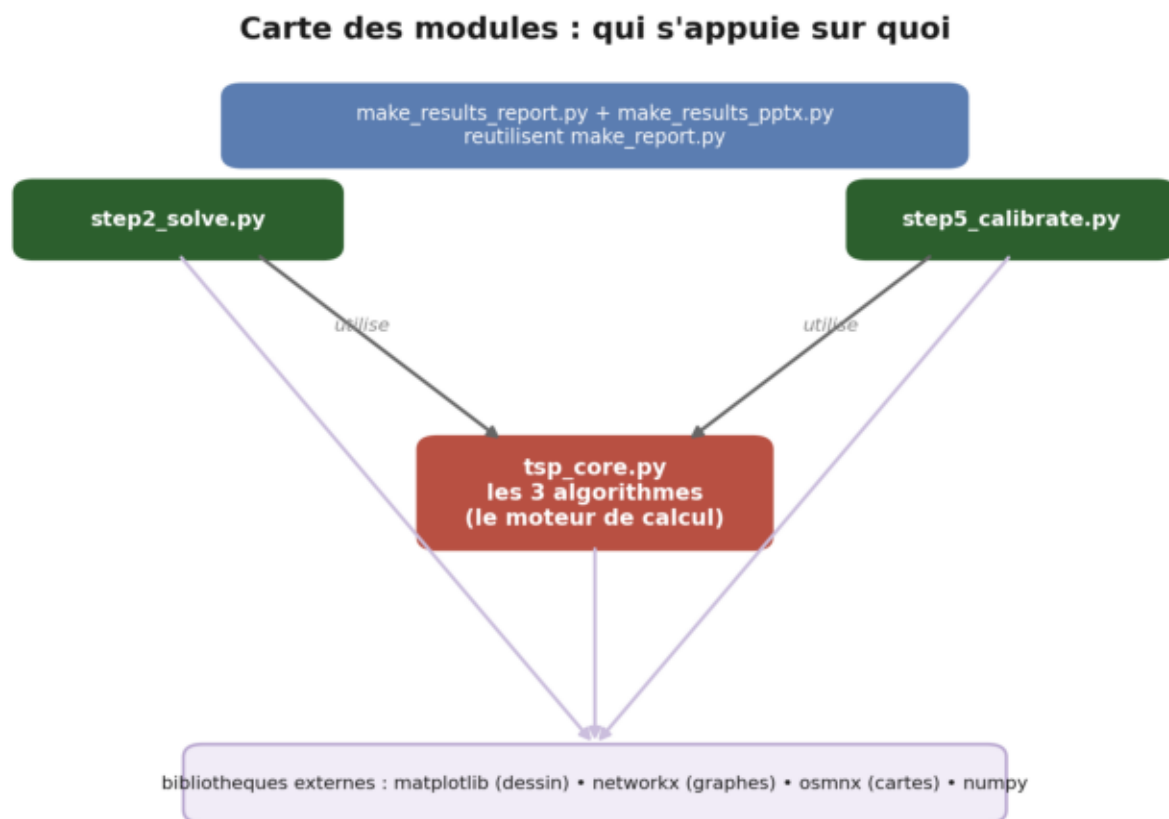
A cote de ce flux principal, des OUTILS analysent et presentent les resultats : step4 (graphe + matrice), step5 (etalonnage), et trois generateurs de rapports.

Schema 2 — Les outils d'analyse et de presentation



step4 et step5 lisent coords.csv. Les generateurs de rapports lisent les fichiers de resultats et produisent les PDF et le PPT.

Schema 3 — Carte des modules (qui utilise quoi)



`tsp_core.py` est le moteur de calcul : `step2` et `step5` s'appuient dessus. Tout le monde s'appuie sur des bibliothèques externes.

2. tsp_core.py — le moteur : les 3 algorithmes

C'est le coeur mathematique. Il ne dessine rien, ne lit pas Internet : il prend une matrice de temps et calcule des tournées. Image : une boite a outils de calcul, utilisee par d'autres fichiers.

read_matrix_csv(path)

A quoi ca sert : lire la matrice des temps depuis un fichier tableur (CSV).

- Entree (ce qu'on lui donne) : le nom du fichier a lire.
- Sortie (ce qu'elle renvoie) : la matrice (tableau de nombres) chargee en memoire.

En clair : elle ouvre le tableur, transforme chaque texte en nombre, et verifie que le tableau est bien carre (autant de lignes que de colonnes).

symmetrize(D)

A quoi ca sert : rendre la matrice symetrique : aller de i a j coute autant que de j a i.

- Entree (ce qu'on lui donne) : une matrice D (eventuellement asymetrique a cause des sens uniques).
- Sortie (ce qu'elle renvoie) : une nouvelle matrice ou chaque case est la moyenne des deux sens.

En clair : $T[i][j]$ devient $(T[i][j] + T[j][i]) / 2$. C'est une hypothese de modelisation, necessaire pour que l'algorithme 2-opt soit valable.

tour_length(tour, D)

A quoi ca sert : mesurer la duree totale d'une tournée donnée.

- Entree (ce qu'on lui donne) : un tour (l'ordre de visite des points) et la matrice des temps D.
- Sortie (ce qu'elle renvoie) : un seul nombre : la somme des temps de toutes les etapes, retour au depart inclus.

En clair : elle additionne le temps de chaque saut consecutif du trajet, puis ajoute le retour du dernier point vers le depart.

nearest_neighbor(D, start=0)

A quoi ca sert : construire une tournée par la regle du PLUS PROCHE VOISIN (algorithme 1).

- Entree (ce qu'on lui donne) : la matrice des temps et le point de depart.
- Sortie (ce qu'elle renvoie) : un tour (liste ordonnee des points).

En clair : depuis le point courant, on va toujours vers le point non encore visite le plus proche, jusqu'a les avoir tous visites. Rapide mais 'myope'.

```
for _ in range(n-1):  
    j = le plus proche non visite  
    on va en j
```

nearest_neighbor_best_start(D)

A quoi ca sert : ameliorer l'algorithme 1 en essayant TOUS les departs.

- Entree (ce qu'on lui donne) : la matrice des temps.
- Sortie (ce qu'elle renvoie) : le meilleur tour obtenu parmi tous les points de depart possibles.

En clair : le plus proche voisin depend du point de depart ; on le relance depuis chaque point et on garde le meilleur resultat.

two_opt(tour, D)

A quoi ca sert : ameliorer une tournee existante par 2-OPT (algorithme 2).

- Entree (ce qu'on lui donne) : un tour de depart et la matrice des temps.
- Sortie (ce qu'elle renvoie) : un tour ameliore (un 'optimum local').

En clair : on cherche les croisements du trajet et on les 'decroise' : on retire deux segments et on les rebranche autrement si cela raccourcit le tour. On repete tant qu'on trouve une amelioration.

held_karp(D)

A quoi ca sert : trouver la tournee EXACTEMENT optimale (algorithme 3, Held-Karp).

- Entree (ce qu'on lui donne) : la matrice des temps (petites instances seulement).
- Sortie (ce qu'elle renvoie) : un couple : le meilleur tour et son cout exact.

En clair : par programmation dynamique : on calcule le meilleur chemin pour chaque petit groupe de points, puis on assemble les grands a partir des petits. Garanti optimal, mais devient impossible au-dela d'environ 20 points.

_subsets(elems, k)

A quoi ca sert : fonction auxiliaire de Held-Karp : lister tous les groupes de k points.

- Entree (ce qu'on lui donne) : une collection d'elements et une taille k.
- Sortie (ce qu'elle renvoie) : tous les sous-groupes possibles de taille k, un par un.

En clair : le tiret bas '_' devant le nom signale une fonction 'interne', utilisee seulement par held_karp, pas destinee a l'utilisateur.

brute_force(D)

A quoi ca sert : verification : essayer TOUTES les tournees possibles (force brute).

- Entree (ce qu'on lui donne) : la matrice des temps (tres petites instances seulement).
- Sortie (ce qu'elle renvoie) : le meilleur tour et son cout.

En clair : elle enumere les $(n-1)!$ ordres possibles. Sert uniquement a verifier que Held-Karp donne bien le meme optimum. Inutilisable au-dela de ~ 10 points.

3. step1_build_matrix.py — fabriquer la matrice des temps

Cette etape transforme tes points GPS en une matrice de temps de trajet, que tout le reste utilisera. Deux methodes possibles.

haversine_m(lat1, lon1, lat2, lon2)

A quoi ca sert : calculer la distance 'a vol d'oiseau' entre deux points GPS.

- Entree (ce qu'on lui donne) : les coordonnees (latitude, longitude) de deux points.

- Sortie (ce qu'elle renvoie) : la distance en metres.

En clair : la Terre est ronde : cette formule (haversine) donne la vraie distance sur la sphere, pas sur une carte plate.

load_points(path)

A quoi ca sert : lire tes points depuis le fichier points.geojson.

- Entree (ce qu'on lui donne) : le nom du fichier de points.
- Sortie (ce qu'elle renvoie) : la liste des coordonnees (longitude, latitude) de chaque point.

En clair : le tout PREMIER point du fichier est considere comme le depot (depart et arrivee de la tournée).

build_matrix_hypothesis(coords)

A quoi ca sert : construire la matrice des temps avec l'HYPOTHESE FORTE.

- Entree (ce qu'on lui donne) : la liste des coordonnees des points.
- Sortie (ce qu'elle renvoie) : la matrice des temps + le temps moyen entre deux points.

En clair : temps = distance a vol d'oiseau / vitesse constante (30 km/h par default). Simple et instantane, mais ignore les vraies routes.

build_matrix_osm(coords)

A quoi ca sert : construire la matrice avec le VRAI reseau routier (mode osm).

- Entree (ce qu'on lui donne) : la liste des coordonnees.
- Sortie (ce qu'elle renvoie) : la matrice des temps reels + sauvegarde du reseau routier.

En clair : elle telecharge les rues autour de tes points (via Internet/OSMnx) et calcule le plus court chemin reel entre chaque paire. Plus fidele, plus lourd.

fmt(sec)

A quoi ca sert : afficher joliment une duree en secondes.

- Entree (ce qu'on lui donne) : un nombre de secondes.
- Sortie (ce qu'elle renvoie) : un texte lisible comme '13min23s'.

En clair : petit utilitaire de presentation, present aussi dans d'autres fichiers.

main()

A quoi ca sert : le chef d'orchestre du fichier : enchainé tout quand on lance step1.

- Entree (ce qu'on lui donne) : rien (il lit les parametres en haut du fichier).
- Sortie (ce qu'elle renvoie) : rien, mais écrit les fichiers de sortie sur le disque.

En clair : il lit les points, choisit la methode, construit la matrice, affiche un resume et sauvegarde travel_time_matrix.csv et coords.csv.

4. step2_solve.py — resoudre et comparer

Cette etape lit la matrice et lance les trois algorithmes pour les comparer. Elle s'appuie entierement sur tsp_core.py.

fmt(sec)

A quoi ca sert : meme utilitaire d'affichage de duree que dans step1.

- Entree (ce qu'on lui donne) : un nombre de secondes.
- Sortie (ce qu'elle renvoie) : un texte lisible.

En clair : duplique ici pour que le fichier reste autonome.

main()

A quoi ca sert : lancer PPV, PPV-meilleur-depart, 2-opt et (si possible) Held-Karp, puis comparer.

- Entree (ce qu'on lui donne) : rien (lit la matrice sur le disque).
- Sortie (ce qu'elle renvoie) : affiche un tableau comparatif et ecrit results.csv et best_tour.json.

En clair : il mesure pour chaque methode le temps de cycle ET le temps de calcul, calcule l'ecart a l'optimum si Held-Karp a tourne, et retient le meilleur tour.

5. step3_plot_map.py — dessiner le meilleur tour

load_coords(path)

A quoi ca sert : relire les coordonnees des points depuis coords.csv.

- Entree (ce qu'on lui donne) : le nom du fichier.
- Sortie (ce qu'elle renvoie) : la liste des points (longitude, latitude).

En clair : necessaire pour savoir ou placer chaque point sur le dessin.

plot_streets(order, coords)

A quoi ca sert : tracer l'itineraire reel le long des rues (si le reseau est disponible).

- Entree (ce qu'on lui donne) : l'ordre de visite et les coordonnees.
- Sortie (ce qu'elle renvoie) : une image best_tour.png suivant les vraies routes.

En clair : utilise le reseau routier sauvegarde par step1 en mode osm.

plot_schema(order, coords)

A quoi ca sert : tracer un schema simple : points relies par des segments droits.

- Entree (ce qu'on lui donne) : l'ordre de visite et les coordonnees.
- Sortie (ce qu'elle renvoie) : une image best_tour.png schematique.

En clair : solution de secours quand le reseau routier n'est pas disponible (comme en mode hypothese) : montre l'ordre de visite, pas les rues.

main()

A quoi ca sert : choisir automatiquement entre rues reelles et schema, puis dessiner.

- Entree (ce qu'on lui donne) : rien.
- Sortie (ce qu'elle renvoie) : l'image best_tour.png.

En clair : s'il existe un reseau routier, il tente les rues ; sinon il retombe sur le schema.

6. step4_graph.py — graphe pondere + matrice d'adjacence

Cet outil represente tes points comme un GRAPHE (des points relies par des traits etiquetes de distances) et affiche la matrice complete.

haversine_m(...)

A quoi ca sert : meme calcul de distance a vol d'oiseau qu'en step1.

- Entree (ce qu'on lui donne) : deux points GPS.
- Sortie (ce qu'elle renvoie) : la distance en metres.

load_coords()

A quoi ca sert : lire les points depuis coords.csv (ou le GeoJSON en secours).

- Entree (ce qu'on lui donne) : rien.
- Sortie (ce qu'elle renvoie) : la liste des points.

distance_matrix(coords)

A quoi ca sert : construire la matrice des distances (en metres) entre tous les points.

- Entree (ce qu'on lui donne) : la liste des points.
- Sortie (ce qu'elle renvoie) : la matrice n x n des distances.

En clair : c'est la matrice d'adjacence du graphe complet : toutes les paires.

label_of(d_m)

A quoi ca sert : formater une distance pour l'ecrire sur une arete.

- Entree (ce qu'on lui donne) : une distance en metres.
- Sortie (ce qu'elle renvoie) : un texte court (metres ou kilometres).

build_graph(coords, D)

A quoi ca sert : construire l'objet graphe (avec networkx) a dessiner.

- Entree (ce qu'on lui donne) : les points et la matrice des distances.
- Sortie (ce qu'elle renvoie) : un graphe ou chaque point est relie a ses plus proches voisins.

En clair : on ne relie que les plus proches voisins pour que le dessin reste lisible.

plot_graph(coords, D, G)

A quoi ca sert : dessiner le graphe : points a leur place reelle, distances sur les traits.

- Entree (ce qu'on lui donne) : les points, la matrice et le graphe.
- Sortie (ce qu'elle renvoie) : l'image graph_view.png.

plot_heatmap(D)

A quoi ca sert : dessiner la matrice complete en carte de chaleur (couleurs).

- Entree (ce qu'on lui donne) : la matrice des distances.
- Sortie (ce qu'elle renvoie) : l'image adjacency_heatmap.png.

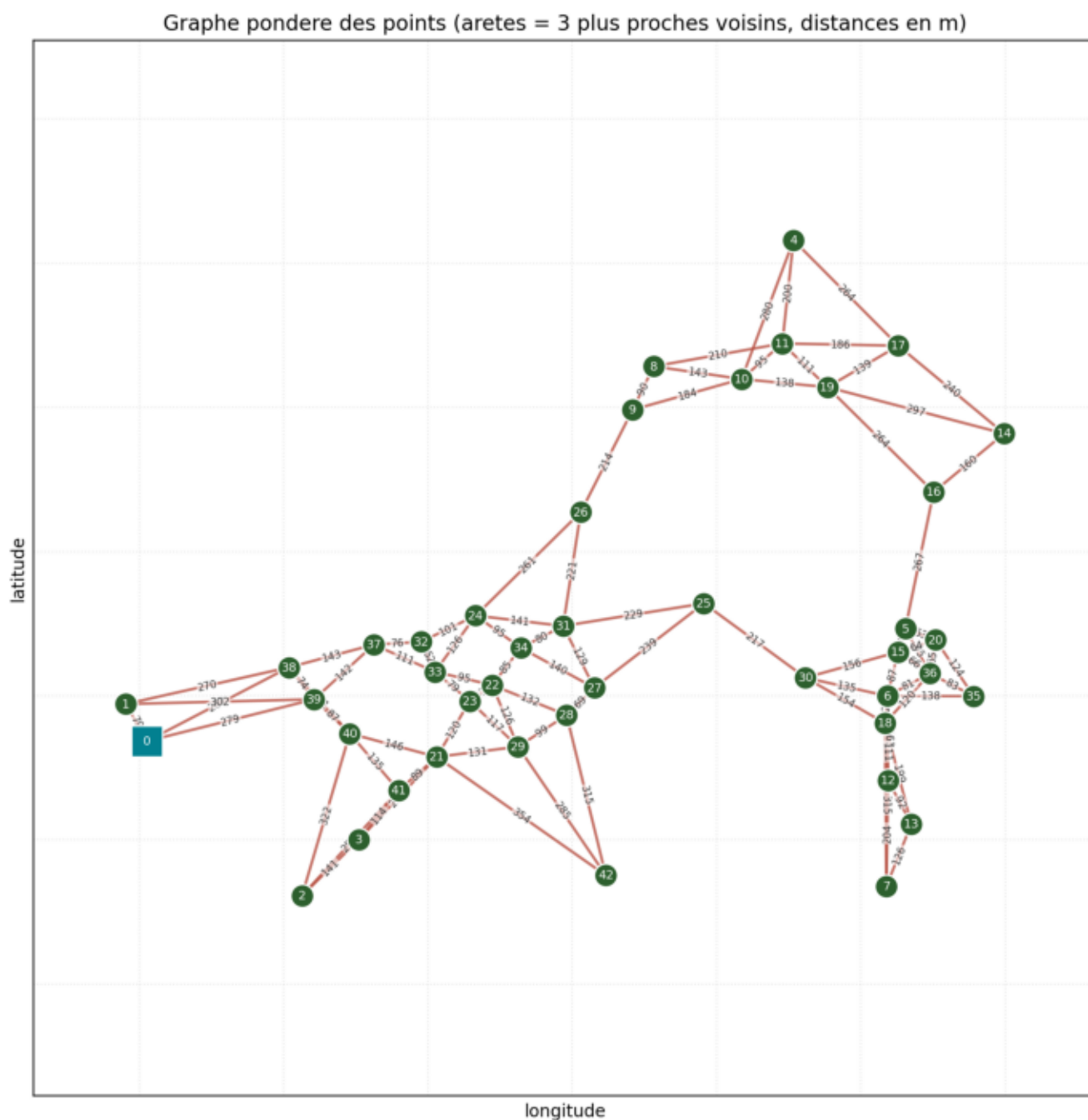
En clair : sombre = proche, clair = lointain ; revele les groupes de points.

save_and_preview_matrix(D)

A quoi ca sert : sauvegarder la matrice en CSV et en montrer un coin a l'ecran.

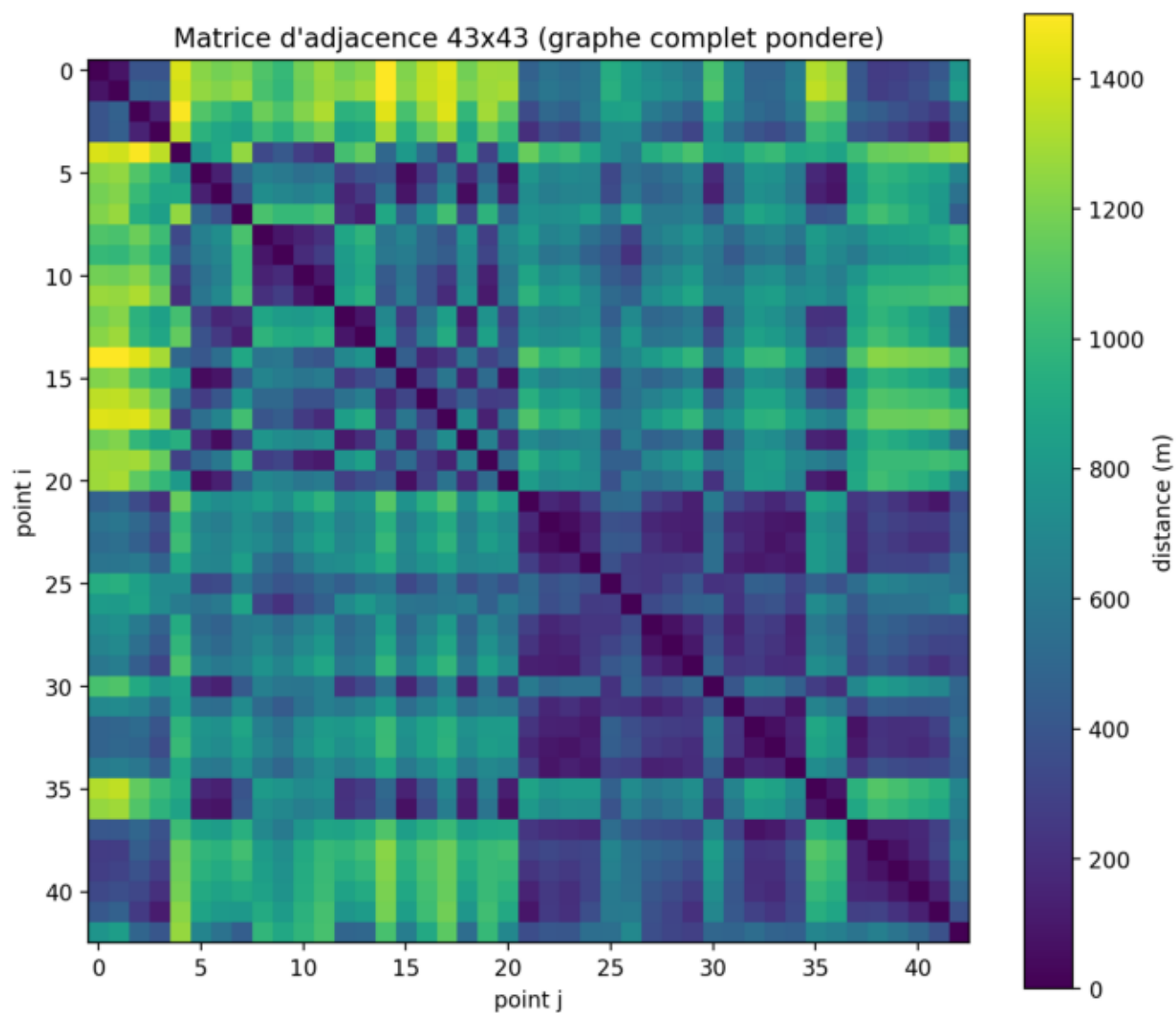
- Entree (ce qu'on lui donne) : la matrice.
- Sortie (ce qu'elle renvoie) : le fichier adjacency_matrix_m.csv + un apercu affiche.

Sortie de step4 — le graphe pondere



Les points a leur position réelle ; chaque trait porte la distance en metres. Le carre est le depot.

Sortie de step4 — la matrice d'adjacence



La meme information, vue comme un tableau de couleurs 43x43.

7. step5_calibrate.py — etalonner les heuristiques

Cet outil mesure la QUALITE des heuristiques en les comparant a l'optimum exact (Held-Karp) sur de petits sous-quartiers.

haversine_m(...) / load_coords()

A quoi ca sert : memes utilitaires que precedemment (distance, lecture des points).

- Entree (ce qu'on lui donne) : points GPS / rien.
- Sortie (ce qu'elle renvoie) : distance / liste de points.

submatrix(coords, idx)

A quoi ca sert : construire la matrice des temps d'un sous-groupe de points.

- Entree (ce qu'on lui donne) : tous les points et la liste des indices choisis.
- Sortie (ce qu'elle renvoie) : la petite matrice des temps de ce sous-quartier.

pick_subquartier(coords, anchor, size)

A quoi ca sert : choisir un sous-quartier coherent : le depot + les voisins d'un point d'ancrage.

- Entree (ce qu'on lui donne) : les points, un point d'ancrage, la taille voulue.
- Sortie (ce qu'elle renvoie) : la liste des indices du sous-quartier (depot en premier).

En clair : on prend les points geographiquement proches de l'ancrage, pour un sous-quartier realiste.

fmt(sec)

A quoi ca sert : afficher une duree lisiblement.

- Entree (ce qu'on lui donne) : des secondes.
- Sortie (ce qu'elle renvoie) : un texte.

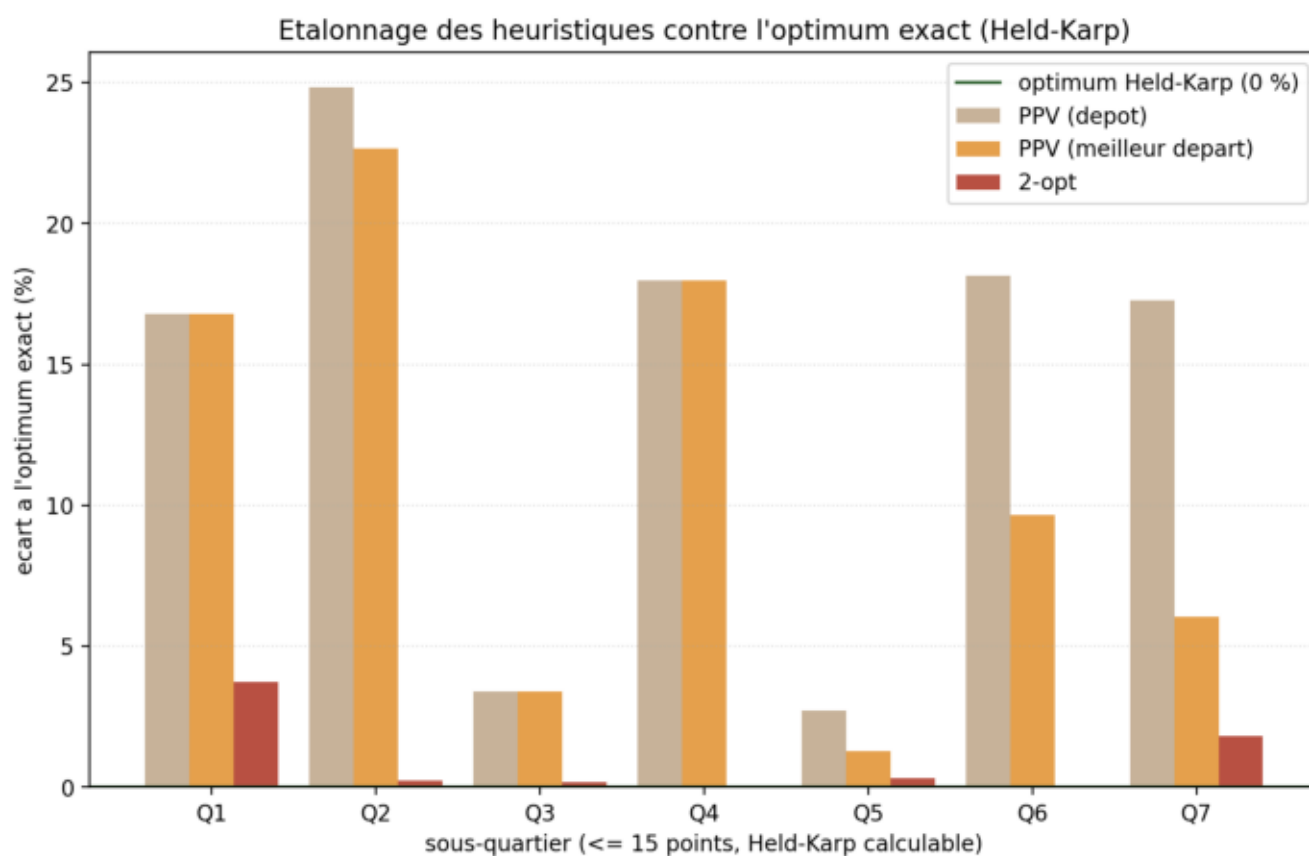
main()

A quoi ca sert : lancer l'etalonnage sur plusieurs sous-quartiers et agreger les ecarts.

- Entree (ce qu'on lui donne) : rien.
- Sortie (ce qu'elle renvoie) : un tableau d'ecarts a l'optimum, calibration_results.csv et calibration_gaps.png.

En clair : pour chaque sous-quartier, il calcule l'optimum exact et mesure de combien de % chaque heuristique s'en ecarte, puis resume la distribution.

Sortie de step5 — l'etalonnage



Ecart a l'optimum exact, par sous-quartier. Le 2-opt (rouge) reste quasiment a 0 % : il est presque toujours optimal.

8. make_report.py — fabriquer le rapport PDF

Ce fichier fabrique un PDF. Il contient la SEULE vraie classe du projet : la classe Doc. On l'explique en detail.

La classe Doc — une 'machine a ecrire des pages'

Rappel : une classe est un appareil qui retient un etat et a des boutons (methodes). La classe Doc retient la page en cours et la position du curseur (ou ecrire la prochaine ligne), et descend automatiquement ; quand la page est pleine, elle en commence une nouvelle.

Ses methodes (ses 'boutons') :

- `__init__` : l'allumage. Cree la machine et prepare la premiere page.
- `_new_page` : commencer une page vierge (et sauver la precedente).
- `_flush` : enregistrer la page courante dans le PDF.
- `_need(h)` : verifier qu'il reste assez de place ; sinon, page suivante.
- `_line` : ecrire une ligne et descendre le curseur.
- `h1 / h2` : ecrire un grand / moyen titre.
- `body` : ecrire un paragraphe (avec retour a la ligne automatique).
- `bullet` : ecrire un point de liste (avec une puce).
- `code` : ecrire un extrait de code en police a chasse fixe.
- `space` : laisser un espace vertical.
- `finish` : enregistrer la derniere page (on a fini d'ecrire).

Les autres fonctions de `make_report.py` :

`image_page(pdf, path, title, caption)`

A quoi ca sert : ajouter une page entiere contenant une image + un titre + une legende.

- Entree (ce qu'on lui donne) : le PDF, le chemin de l'image, un titre, une legende.
- Sortie (ce qu'elle renvoie) : une page ajoutee au PDF.

`read_results() / n_points()`

A quoi ca sert : lire le tableau de resultats / compter les points.

- Entree (ce qu'on lui donne) : rien.
- Sortie (ce qu'elle renvoie) : les donnees a afficher dans le rapport.

`cover(pdf) / main()`

A quoi ca sert : dessiner la couverture / assembler tout le rapport.

- Entree (ce qu'on lui donne) : le PDF.
- Sortie (ce qu'elle renvoie) : le fichier `rapport_tipe.pdf` complet.

9. make_results_report.py & make_results_pptx.py

Ces deux fichiers presentent les RESULTATS (instance 43 points + etalonnage). Le premier produit un PDF, le second un diaporama PowerPoint. Tous deux relisent `results.csv` et

calibration_results.csv et reutilisent des briques de make_report.py.

fmt / read_results / read_calibration / read_csv

A quoi ca sert : utilitaires : formater une duree, lire les fichiers de resultats.

- Entree (ce qu'on lui donne) : des secondes / des noms de fichiers.
- Sortie (ce qu'elle renvoie) : un texte lisible / des tableaux de donnees.

cover / title slide

A quoi ca sert : fabriquer la page (ou diapo) de titre.

- Entree (ce qu'on lui donne) : le document.
- Sortie (ce qu'elle renvoie) : la couverture.

results_table / calib_table

A quoi ca sert : dessiner les tableaux de chiffres (resultats et ecarts).

- Entree (ce qu'on lui donne) : les donnees lues.
- Sortie (ce qu'elle renvoie) : un tableau mis en forme dans la diapo.

add_image_fit / image_page / caption / bullets

A quoi ca sert : placer une image au bon format, ajouter une legende, ecrire des puces.

- Entree (ce qu'on lui donne) : image et textes.
- Sortie (ce qu'elle renvoie) : des elements ajoutes a la page/diapo.

main()

A quoi ca sert : assembler le livrable complet (PDF ou PPTX).

- Entree (ce qu'on lui donne) : rien.
- Sortie (ce qu'elle renvoie) : resultats_tipe.pdf ou Resultats_TIPE_13530.pptx.

10. Recapitulatif : quel fichier pour quoi

- tsp_core.py : les 3 algorithmes (le calcul pur).
- step1 : points -> matrice des temps.
- step2 : matrice -> comparaison des algorithmes.
- step3 : meilleur tour -> carte.
- step4 : points -> graphe + matrice d'adjacence.
- step5 : etalonnage des heuristiques contre l'optimum exact.
- make_report : documentation/rapport PDF (contient la classe Doc).
- make_results_report / make_results_pptx : presentation des resultats (PDF + PPT).
- make_doc : CE document.